

Secondo Assignment - Dataflow Analysis

Corso: Compilatori – Middle end

Studente: Gnaccarini, D'amato, Garuffi

1) Very Busy Expressions

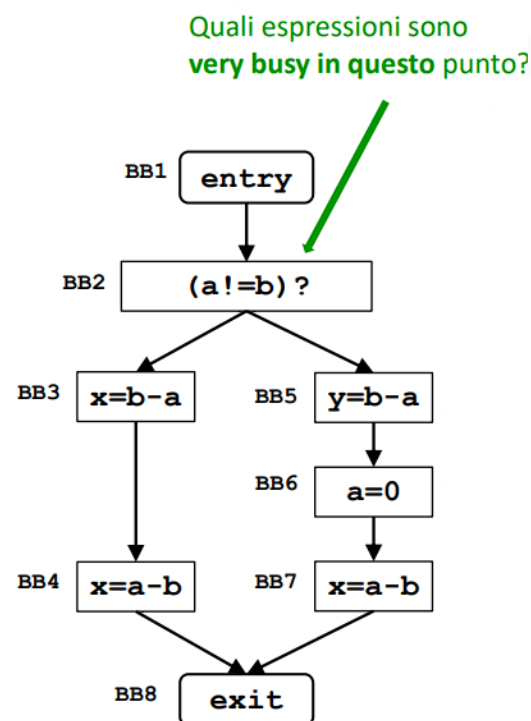
Un'espressione è very busy in un punto p se, indipendentemente dal percorso preso da p , l'espressione viene usata prima che uno dei suoi operandi venga definito.

Un'espressione $a+b$ è very busy in un punto p se $a+b$ è valutata in tutti i percorsi da p a EXIT

- Eventuali ridefinizioni di a o b lungo tali percorsi sono presenti solo dopo la valutazione
- Ci interessa l'insieme di espressioni disponibili (available) all'inizio del blocco B
- L'insieme dipende dai percorsi che cominciano al punto p prima di B

1. Formalizzazione del Framework

Dataflow Problem	Very Busy Expressions
Domain	Set of expression
Direction	backward: $in[b] = f_b(out[b])$ $out[b] = \bigwedge in[succ(b)]$
Transfer function	$f_b(x) = Gen_b \cup (x - Kill_b)$
Meet Operation (^)	$\cap$
Boundary Condition	$in[exit] = \emptyset$
Initial interior points	$in[b] = \cup$



2. Tabella delle Iterazioni

Basic Block	Iterazione 1	
	IN	OUT
BB1	-	b-a
BB2	b-a	b-a
BB3	b-a, a-b	a-b
BB4	a-b	$\emptyset$
BB5	b-a	$\emptyset$
BB6	$\emptyset$	a-b
BB7	a-b	$\emptyset$
BB8	$\emptyset$	-

Risposta:

{b-a}, poiché è l'unica espressione utilizzata da BB2 a EXIT prima che uno dei suoi operandi venga definito.

Ricordiamo che:

$$in[b] = out[pred[b]] \cap out[pred[b]] \cap \dots$$

$$out[b] = b \cup \{in[b]\}$$

2) Dominator Analysis

In un CFG diciamo che un nodo X domina un altro nodo Y se il nodo X appare in ogni percorso del grafo che porta dal blocco ENTRY al blocco Y

- Annotiamo ogni basic block B_i con un insieme $DOM[B_i]$
- $B_i \in DOM[B_j]$ se e solo se B_i domina B_j
- Per definizione un nodo domina sé stesso
- $B_i \in DOM[B_i]$

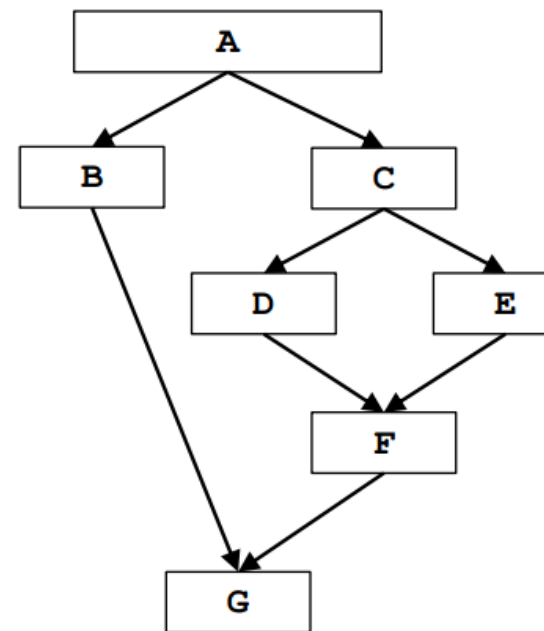
1. Formalizzazione del Framework

Dataflow Problem	Dominator Analysis
Domain	set of bb
Direction	forward $in[B] = \Lambda$ out[pred(b)] $out[B] = f_b(in[b])$
Transfer function	$f_b(x) = B \cup (prec[B])$
Meet Operation ($\wedge$)	intersezione
Boundary Condition	$out[A] = A$
Initial interior points	$out[b] = \cup$

Ricordiamo che:

$$in[b] = out[pred[b]] \cap out[pred[b]] \cap \dots$$

$$out[b] = b \cup \{in[b]\}$$



$$DOM[F] = \{A, C, F\}$$

2. Tabella delle Iterazioni

Basic Block	Iterazione 1	
	IN	OUT
A	-	A
B	A	B,A
C	A	C,A
D	A,C	A,C,D
E	A,C	A,C,E
F	A,C	A,C,F
G	A	-

Come si evince dalla formalizzazione del framework, l'insieme dei dominatori (DOM) di un nodo corrisponde all'insieme OUT calcolato dall'analisi dataflow.

Non è stato necessario definire esplicitamente gli insiemi GEN e KILL, in quanto la funzione di trasferimento lavora direttamente a livello di Basic Block, limitandosi all'unione del blocco stesso (B) con le informazioni provenienti dai suoi predecessori.

3) Constant Propagation

L'obiettivo della constant propagation è quello di determinare in quali punti del programma le variabili hanno un valore costante.

- L'informazione da calcolare per ogni nodo n del CFG è un insieme di coppie del tipo $\langle \text{variabile}, \text{valore costante} \rangle$.
- Se abbiamo la coppia $\langle x, c \rangle$ al nodo n , significa che x è garantito avere il valore c ogni volta che n viene raggiunto durante l'esecuzione del programma.

1. Formalizzazione del Framework

Dataflow Problem	Constant Propagation
Domain	set of pairs $\langle x, c \rangle$
Direction	Forward: $\text{in}[b] = \bigwedge \text{out}[\text{pred}(b)]$ $\text{out}[b] = f_b(\text{in}[b])$
Transfer function	$f_b(x) = \text{gen}_b \vee (x - \text{kill}_b)$
Meet Operation (^)	$\cap$
Boundary Condition	$\text{out}[\text{entry}] = \emptyset$
Initial interior points	$\text{out}[b] = \emptyset$

Ricordiamo che:

$$\text{in}[b] = \text{out}[\text{pred}[b]] \cap \text{out}[\text{pred}[b]] \cap \dots$$

$$\text{out}[b] = \text{Gen}[b] \cup \{\text{in}[b] - \text{Kill}[b]\}$$

2. Tabella delle Iterazioni

Basic Block	Iterazione 1		Iterazione 2	
	IN	OUT	IN	OUT
ENTRY	-	$\emptyset$	-	$\emptyset$
K = 2	$\emptyset$	$\{(k,2)\}$	$\emptyset$	$\{(k,2)\}$
if	$\{(k,2)\}$	$\{(k,2)\}$	$\{(k,2)\}$	$\{(k,2)\}$
a = k+2	$\{(k,2)\}$	$\{(k,2), (a,4)\}$	$\{(k,2)\}$	$\{(k,2), (a,4)\}$
x = 5	$\{(k,2), (a,4)\}$	$\{(k,2), (a,4), (x,5)\}$	$\{(k,2), (a,4)\}$	$\{(k,2), (a,4), (x,5)\}$
a = k*2	$\{(k,2)\}$	$\{(k,2), (a,4)\}$	$\{(k,2)\}$	$\{(k,2), (a,4)\}$
x = 8	$\{(k,2), (a,4)\}$	$\{(k,2), (a,4), (x,8)\}$	$\{(k,2), (a,4)\}$	$\{(k,2), (a,4), (x,8)\}$
k = a	$\{(k,2), (a,4)\}$	$\{(k,4), (a,4)\}$	$\{(k,2), (a,4)\}$	$\{(k,4), (a,4)\}$
while	$\{(k,4), (a,4)\}$	$\{(k,4), (a,4)\}$	$\{(a,4)\}$	$\{(a,4)\}$
b = 2	$\{(k,4), (a,4)\}$	$\{(k,4), (a,4), (b,2)\}$	$\{(a,4)\}$	$\{(a,4), (b,2)\}$
x = a + k	$\{(k,4), (a,4), (b,2)\}$	$\{(k,4), (a,4), (b,2), (x,8)\}$	$\{(a,4), (b,2)\}$	$\{(a,4), (b,2)\}$
y = a*b	$\{(k,4), (a,4), (b,2), (x,8)\}$	$\{(k,4), (a,4), (b,2), (x,8), (y,8)\}$	$\{(a,4), (b,2)\}$	$\{(a,4), (b,2), (y,8)\}$
k++	$\{(k,4), (a,4), (b,2), (x,8), (y,8)\}$	$\{(k,5), (a,4), (b,2), (x,8), (y,8)\}$	$\{(a,4), (b,2), (y,8)\}$	$\{(a,4), (b,2), (y,8)\}$
print(A+k)	$\{(k,4), (a,4)\}$	$\{(k,4), (a,4)\}$	$\{(a,4)\}$	$\{(a,4)\}$
EXIT	$\{(k,4), (a,4)\}$	-	$\{(a,4)\}$	-

